



COURSE TITLE: DESIGN PROJECT-1

COURSE CODE: CSE 2200

Section: 01

***COMPLEX ENGINEERING PROJECT
FINAL REPORT***

Project Name: Central Banking System

SUBMITTED BY: GROUP-4

<i>Name</i>	<i>Student ID</i>
<i>Md Fuad Anan</i>	<i>251014032</i>
<i>Tapu Shikari</i>	<i>222014064</i>
<i>Sithi Saha</i>	<i>242014064</i>
<i>Jannatul Ferdous Lamia</i>	<i>242014001</i>
<i>Maha Islam</i>	<i>252014045</i>

SUBMITTED TO:

Md Al Amin Sheikh, Ph.D.

Assistant Professor

Department of Computer Science and Engineering (CSE)

University of Liberal Arts Bangladesh

Date: 19 August 2026

Central Banking System

Abstract:

The Central Banking System Simulator is a Java-based web application developed to demonstrate how a central bank can manage and monitor different banking activities from one system. The project includes three main user roles: Governor, Administrator, and Commercial Bank. Each role has separate access and responsibilities so that the system can maintain a proper hierarchy and controlled operation.

The system supports commercial bank registration and management, customer and account handling, loan processing, inter-bank fund transfer, deposits and withdrawals, reserve management, monetary policy updates, money supply and liquidity operations, approval requests, notifications, and report generation. It also includes risk assessment and rule-based fraud detection for monitoring banking activities. The Governor can review important requests and manage administrators, while the Administrator and Commercial Bank users can perform operations according to their permission level.

The application is developed using Java, HTML, and CSS. Java's built-in HTTP server is used to run the web interface locally, and CSV files are used for storing data such as banks, customers, accounts, loans, transactions, reserves, policies, exchange rates, and logs. The system also displays USD and EUR exchange-rate information from an external API, with stored data available when a live request cannot be completed.

This project is a simulator, not a real central banking platform. Its purpose is to provide a controlled environment for understanding central banking operations, applying object-oriented programming concepts, and practicing the management and monitoring of banking activities without affecting a real financial system.

Table of Contents

Serial No.	Contents	Pg No.
1.	Introduction	4
2.	Objectives	4
3.	Background Study	5
4.	Survey Questionnaires	5-6
5.	Response Summary	6
6.	Problem and Solution Description	6-7
7.	Project Scope	7-8
8.	Feasibility Analysis	9-10
9.	Technical Requirements	11-12
10.	Functional Requirements	12
11.	Non-Functional Requirements	13
12.	System Design	14-17
13.	Development Methodology	18-23
14.	Implementation	23-27
15.	Conclusion	28
16.	Future Work & Limitations	28-29
17.	References	29

Introduction:

A central bank is an institution that manages the monetary policy of a country or monetary union. In contrast to a commercial bank, a central bank possesses a monopoly on increasing the monetary base.^[1] It makes sure commercial banks follow the rules, controls the money supply and maintains financial stability. The central bank also has to be transparent, watch over banking activities and protect the system from cyber threats and security risks. Many computerized systems are used to manage banking operations, such as processing transactions, managing accounts and reporting. However, a lot of these systems are complicated, hard to monitor and difficult to manage. This makes it difficult to monitor and supervise multiple banking operations simultaneously. In Bangladesh, Bangladesh Bank has started digital initiatives, like electronic fund transfer systems, real-time payment settlement mechanisms and mobile financial services. These changes show how important digital technologies are in banking operations. Automation in banking is necessary because it reduces mistakes, speeds up processing, makes things more transparent and helps with decision-making. That is why developing a Central Banking System Simulator is a valuable initiative. This simulator can give people an idea of banking operations, test processes and train users without affecting the actual banking system. This is especially important for institutions like Bangladesh Bank, where monitoring and managing banking activities efficiently crucial. The Central Banking System Simulator will be very useful for Bangladesh Bank. Bangladesh Bank can use the simulator to understand banking operations. The simulator will help Bangladesh Bank test processes and train its users. This will make Bangladesh Bank more efficient and effective. The Central Banking System Simulator is a tool for Bangladesh Bank and other central banks. It will help them improve their operations and provide services to people.

Objectives:

The main objective of our project is to design and develop a Central Banking System Simulator for managing and monitoring banking operations.

Specific mini goals of our project are:

1. Make a System where multiple banks are registered and managed under Central banking System.
2. Do money supply control operations to keep economy running.
3. Monitor banking transactions between commercial banks so things like fraud and money laundering does not happen.
4. Manage national reserve's information so nation does not face financial crises.
5. Give a smooth user-friendly graphical interface so user can be comfortable using it
6. Generate reports about banks and their risks in investment.

Background Study:

Before developing the Central Banking System Simulator, we studied the basic responsibilities of a central bank and how it works with commercial banks. A central bank is responsible for maintaining financial stability, controlling money supply, setting monetary policies, managing reserves, and supervising commercial banks. Unlike normal commercial banks, a central bank mainly works as a regulator and controller of the banking environment.

We studied how commercial banks need to be monitored through information such as their reserve position, loans, transactions, financial condition, and risk level. Reserve management is important because it helps a country maintain financial stability during difficult economic situations. We also studied monetary policy concepts such as interest rate, reserve requirement, and liquidity management. These concepts helped us understand how a central bank can influence banking activities and the flow of money in the economy.

Another important part of our study was transaction monitoring. In a real banking environment, unusual transactions, repeated failed transfers, very large transfers, and weak financial conditions may need attention. For this reason, our system includes a simple fraud detection process and risk assessment process. The risk assessment considers factors such as reserve position, debt ratio, credit rating, previous loans, and requested loan amount.

We also studied the importance of role-based access in banking systems. A system should not give the same control to every user. In our project, the Governor has the highest authority, the Administrator manages banking operations, and Commercial Banks can access their own related functions. This study helped us design separate dashboards and permissions for each role.

We used Bangladesh Bank official resources and other central banking-related materials mentioned in the reference section to understand the basic banking concepts. The purpose of this background study was not to copy a real central bank system, but to create a simplified simulator that represents important central banking operations in an understandable way.

Survey Questionnaires:

For survey we sent email asking the questions below to some famous central banks like Bangladesh Central Bank, Europe Central Bank, India Central Bank and commercial banks.

1. How user-friendly do you find the overall design and interface of the website?
2. Are the features (such as bank registration, risk monitoring, and loan management) clear and easy to understand?
3. Do you think the system includes all necessary functionalities required for a central banking system? If not, what additional features would you suggest?

4. How effective is the risk monitoring section (liquidity tracking, capital adequacy, etc.)?
5. Is the navigation of the website smooth and intuitive?
6. Are there any difficulties or issues you experienced while using the website?
7. How can we improve the performance, design, or usability of the system?
8. Would you consider this system useful for practical implementation in banking operations? Why or why not?

Response Summary:

Although we sent many survey email but no one responded and only got a reply from Europe Central Bank also that was a polite decline.

Dear Fuad,

Thank you for your email.

I am afraid that, as a matter of policy, the Bank does not partake in surveys, interviews, or questionnaires. This is due to the time constraints of our staff and the number of requests we receive. As such, it is not possible to fulfill your request.

Wishing you and your group all the best with your studies.

Kind regards,



Helena
Engagement and Enquiries
Bank of England

Threadneedle Street, London EC2R 8AH
enquiries@bankofengland.co.uk
+44 20 3461 4878

[Privacy notice](#)

When you contact us, the Bank of England collects information about you. This includes your name, contact details and anything you choose to disclose in your correspondence.

We use your personal data to manage correspondence and (where applicable) to respond to you. Our basis to process this data is that it is necessary for the performance of a task carried out in the public interest or in the exercise of official authority of the Bank.

We will keep your personal data for 10 years. You can request that we no longer use your personal data, by contacting us via the website link below.

To find out more about the Bank's approach to privacy, your data protection rights, to make a complaint, or to contact our Data Protection Officer, please see our website at www.bankofengland.co.uk/privacy

Please note, a copy of all incoming correspondence and enclosures where practicable will be filed electronically in the Bank of England's record management system in accordance with its standard record management policy and statutory obligations. If you become aware of any copyright work that you consider should not be filed in this way, please let us know and we will review whether it may be excluded from electronic filing.

We strive to be as open and accessible as possible, and are aware some people may wish to contact us in different ways. If you require any adjustments please do let us know. You can also read more about our accessibility policy [here](#).

Problem and Solution Description:

1. Problem Description

Managing the activities of a central bank involves a large amount of financial and administrative information. Commercial banks need to be monitored regularly, and their financial condition, loans, transactions and reserve information need to be checked.

Some major problems are:

- Difficulty in managing information from multiple banks.
- Lack of centralized monitoring.
- Manual handling of approval requests.

- Difficulty in monitoring suspicious transactions.
- Difficulty in maintaining loan records.
- Difficulty in maintaining reserve information.
- Lack of proper role-based access.
- Difficulty in generating financial reports.
- Risk of inconsistent records when information is maintained separately.

2. Proposed Solution

Our Central Banking System provides a centralized web-based platform to manage these activities efficiently with role-based dashboards for different user types (Governor, Administrator, and Commercial Bank).

The proposed system provides:

- **Role-based access & dedicated dashboards** (Governor, Administrator, Commercial Bank)
- Commercial bank management
- Customer management
- Account management
- Loan management
- Inter-bank transactions
- Reserve management
- Monetary policy management
- Money supply & liquidity operations
- Risk assessment
- Fraud detection & suspicious activity monitoring
- Approval request management
- Real-time notifications
- Exchange-rate information & conversion
- Financial report generation
- System logging (Login, Activity, and Audit logs)

The system reduces the need to manage these activities separately and provides a structured way to monitor the banking environment.

Project Scope:

The scope of the project includes the following areas:

1. Bank Management

The system can register commercial banks, edit bank information, suspend banks, and process bank-related requests.

The sample project data contains three commercial banks representing different banking sectors:

- **Delta Commercial Bank** (Commercial Bank)
- **Padma Islamic Bank** (Islamic Bank)
- **Meghna Development Bank** (Development Bank)

2. Customer Management

Commercial banks can add and edit customer information. The system stores customer details including **Customer Name, Email, Phone Number, NID, Address, Bank ID, Status, and Account ID**.

3. Account Management

The system maintains account information including account ID, customer ID, bank ID, account type, balance, and active status.

4. Loan Management

Commercial banks can apply for loans. Loans can have different types (e.g., Short-term, Long-term) and statuses such as **Pending, Approved, Disbursed, or Rejected**.

5. Transaction Management

The system supports inter-bank transfers between commercial banks. Transactions record transaction ID, sender bank, receiver bank, amount, date/time, transaction type, status, and description.

6. Reserve Management

The system stores national reserve information and minimum required reserve. It can automatically evaluate and determine whether a bank's reserve falls below the required threshold.

7. Monetary Policy

The system maintains policy information such as interest rate, reserve requirement percentage, effective dates, and policy descriptions.

8. Risk Assessment

The system calculates a comprehensive risk score using factors such as reserve ratio, debt ratio, credit rating, previous loan history, and requested loan amount.

9. Reporting

The system can generate different structured reports including:

- Bank report
- Customer report
- Transaction report
- Reserve report
- Fraud report
- Loan report
- Investment risk report
- Performance report
- Summary report

Feasibility Analysis:

1. Technical Feasibility

The project is technically feasible as it is built using Java and standard Java libraries. The project utilizes **Apache Maven (pom.xml)** for build management and can be compiled using **compile.bat**, tested using **test.bat**, and executed using **run.bat**.

In the system we use Java's built-in HTTP server (**com.sun.net.httpserver**) for rendering the web interface and structured CSV files for lightweight data persistence.

The source code strictly follows object-oriented programming (OOP) principles with separate entities, services, and routing layers.

2. Operational Feasibility

The system is designed for three major user categories: **Governor**, **Administrator**, and **Commercial Bank**.

Each user receives a specialized dashboard and specific permission levels. This improves operational feasibility as users only interact with functions relevant to their designated responsibilities.

3. Economic Feasibility

The project was developed using open-source technologies and standard Java tools. No expensive commercial software licenses are required to run or maintain the system.

Estimated Project Budget:

Items	Costs
Development software	0.00 BDT
Java/JDK	0.00 BDT
Code Editor/IDE (Apache Netbeans)	0.00 BDT
Testing Tools	0.00 BDT
Internet and Miscellaneous Expenses	1000.00 BDT
Documentation/printing, Electricity Bill	500.00 BDT
Total Cost	1500.00 BDT

4. Project Timeline

A structured development timeline for the project is:

Phase	Work
Week 1-3	Choosing Project topic and make Project proposal
Week 4	Requirement analysis
Week 5	Background Study on Central Bank
Week 6	Working on initial UML Diagram and other Diagram
Week 7	Learn how web application works and apply them and self-host webpage
Week 8	Make a home page and Add Admin login function and Commercial bank Login function
Week 9	Do Backend code behind functions and connect them
Week 10	Add live exchange rate feature
Week 11	Add Governor and functions of governor.
Week 12	Update other functions with governor and debug the code.
Week 13	Update the Use Case Diagram, UML Class Diagram, Activity Diagram and Sequence Diagram according to finished project
Week 14	Final report and presentation

Technical Requirements:

The Central Banking System Simulator is a Java application that runs on its own. It is designed to be easy to use and understand. The system is made up of parts that work together to make it run smoothly. The Central Banking System Simulator uses Java as its main programming language and CSS and html are also use in it. It uses the Java Development Kit which's a set of tools that helps us build the system. The system is designed using object-oriented programming principles, which means it is made up of objects that work together to get things done.

1. Programming Language and Core Paradigms

We use the Java Development Kit to build the system. It is like a set of tools that helps us create the system. The system is designed using object-oriented programming principles, which include encapsulation, inheritance, polymorphism and abstraction.

2. Architecture and UI Engine

The system has a built-in web server that serves up web pages to users. It is like a website that users can interact with. The system also has a custom web page renderer that makes the web pages look nice and work well.

3. Data Persistence and File Management

The system stores its data in text files. It uses these files to store all of the systems data. The system also has tools that help it read and write data to these files.

4. Development. Build Automation

We use a variety of tools to build and test the system. These tools include Apache Maven, which helps us manage the systems dependencies and build the system. We also use Windows Batch Scripts to automate tasks.

5. System Data Architecture

The system stores its data in text files. These files include data about accounts, banks, transactions, loans and more.

Accounts.csv: This file stores data about accounts including the account number, bank ID, customer ID, account type, balance, currency and status.

Banks.csv: This file stores data about banks, including the bank ID, name, swift code, bank type, status, total reserve and capital.

Transactions.csv: This file stores data about transactions, including the transaction ID, source account, target account, amount, currency, transaction type, timestamp and status.

Loans.csv: This file stores data about loans, including the loan ID, customer ID, bank ID, principal amount, interest rate, loan type and status.

Reserve.csv: This file stores data about reserves including the reserve ID, bank ID required ratio, actual cash balance and deficit or surplus.

Policies.csv: This file stores data about policies, including the policy ID, policy name, repo rate reverse repo rate CRR rate, SLR rate and effective date.

Approval_requests.csv: This file stores data about approval requests, including the request ID, request type requesting bank ID, details, status and submitted timestamp.

Exchange_rates.csv: This file stores data about exchange rates, including the currency pair, buying rate, selling rate and last updated timestamp.

Activity_logs.csv / audit_logs.csv: These files store data about system activity, including the log ID, user role action executed, IP or session ID, timestamp and security result.

Functional Requirements:

The Central Banking System Simulator has different functions that it can perform. These functions include role-based authentication and session management, commercial bank supervision and registration, reserve management and statutory liquidity control and more.

1. Role-Based Authentication and Session Management
2. Commercial Bank Supervision and Registration
3. Reserve Management and Statutory Liquidity Control
4. Monetary Policy Implementation and Liquidity Operations
5. Fraud Detection and Interbank Transaction Monitoring
6. Customer Account and Loan Supervision
7. Foreign Exchange Rate Management
8. Automated Regulatory Reporting and Notifications

Non-Functional Requirements:

Non-functional requirements spell out the quality, performance, security, usability, and reliability standards the Central Banking System needs to uphold while it runs. Here are the main ones.

1. Security
2. Reliability
3. Performance
4. Usability
5. Integrity of Data
6. Accessibility
7. Maintainability
8. Adaptability
9. The power to audit
10. Managing Errors
11. Expandability
12. Testability
13. Transportability
14. Privacy
15. General Quality

Activity Diagram

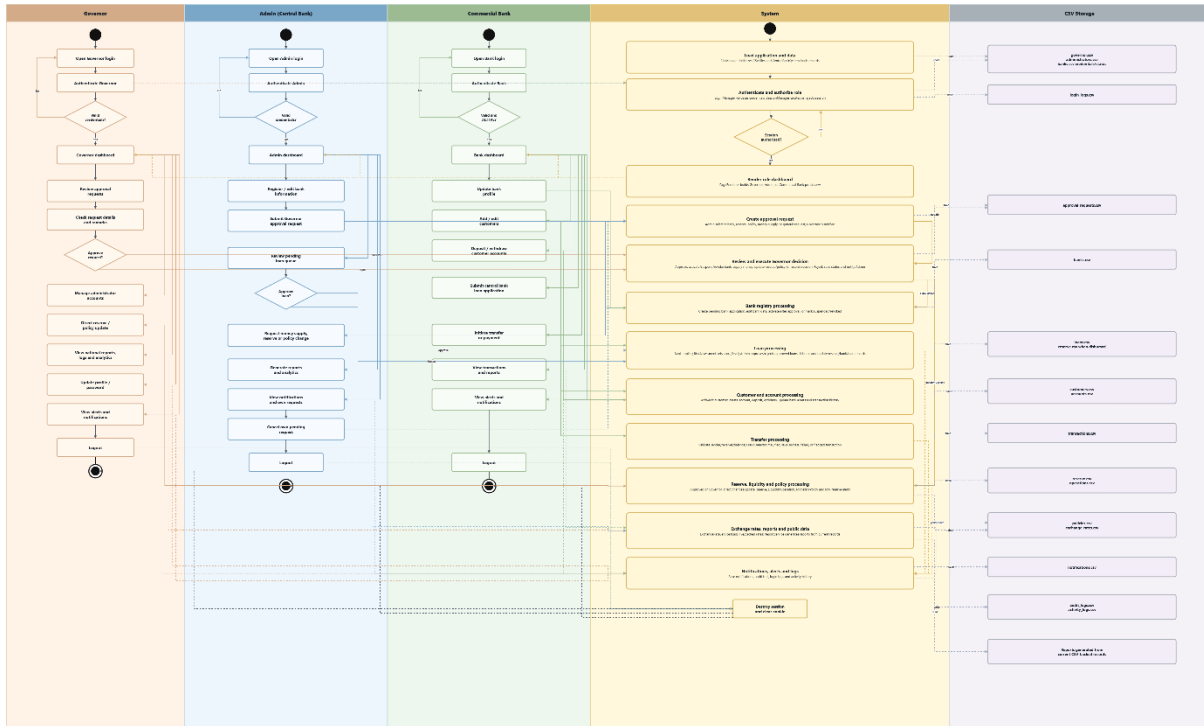


Figure 01: Activity Diagram of Central Banking System

The Activity Diagram shows the step-by-step workflow of the Central Banking System. It explains how activities such as login, bank management, loan processing, transactions, approvals, and report generation move from one step to another.

Use Case Diagram:

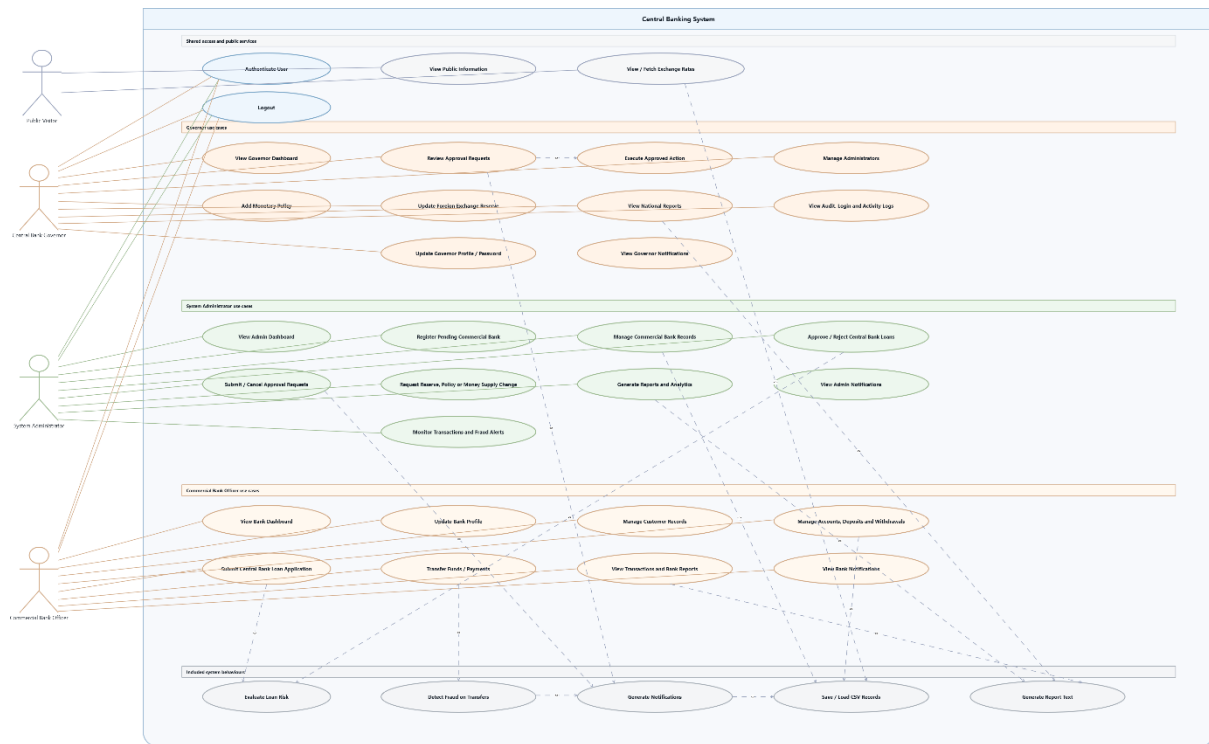


Figure 02: Use-Case Diagram of Central Banking System

The Use Case Diagram shows the main users (Governor, Administrator, and Commercial Bank) and the functions they can perform in the system. It gives an overall view of how each actor interacts with the Central Banking System.

UML Diagram:

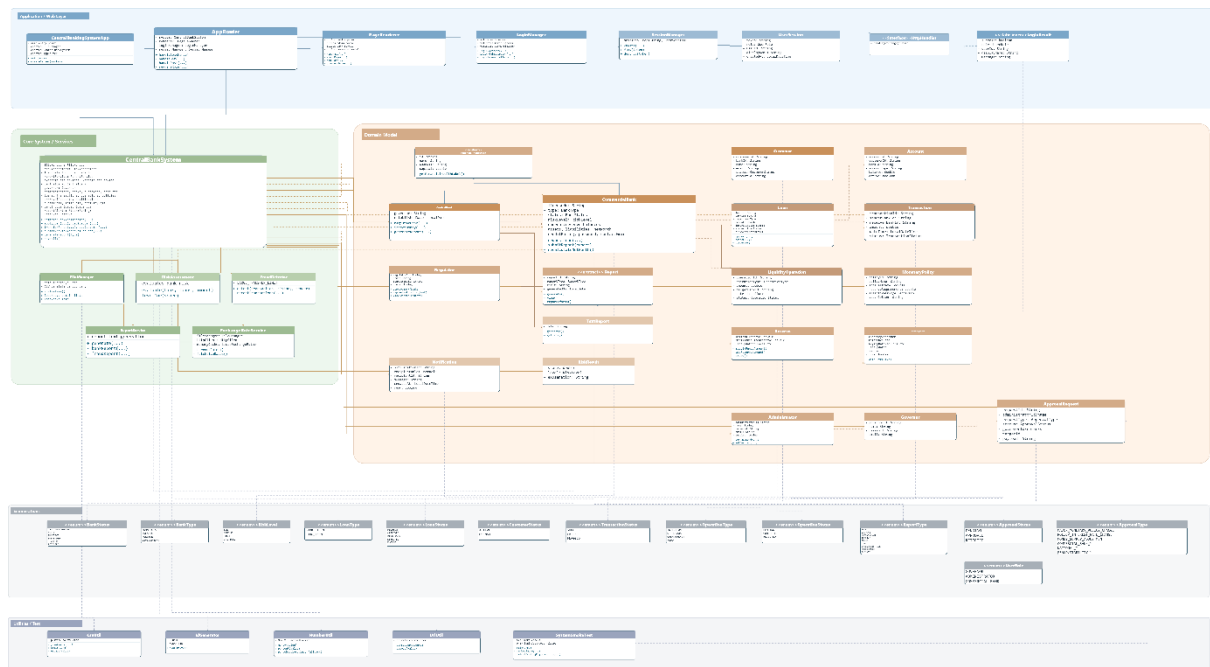


Figure 03: UML Class Diagram of Central Banking System

The UML Class Diagram shows the internal structure of the system. It represents important classes such as CentralBank, CommercialBank, Customer, Account, Loan, Transaction, Reserve, MonetaryPolicy, ApprovalRequest, and Report, along with their attributes, methods, and relationships. Because of bug in editing software the picture's writing is not in place.

System Sequence Diagram:

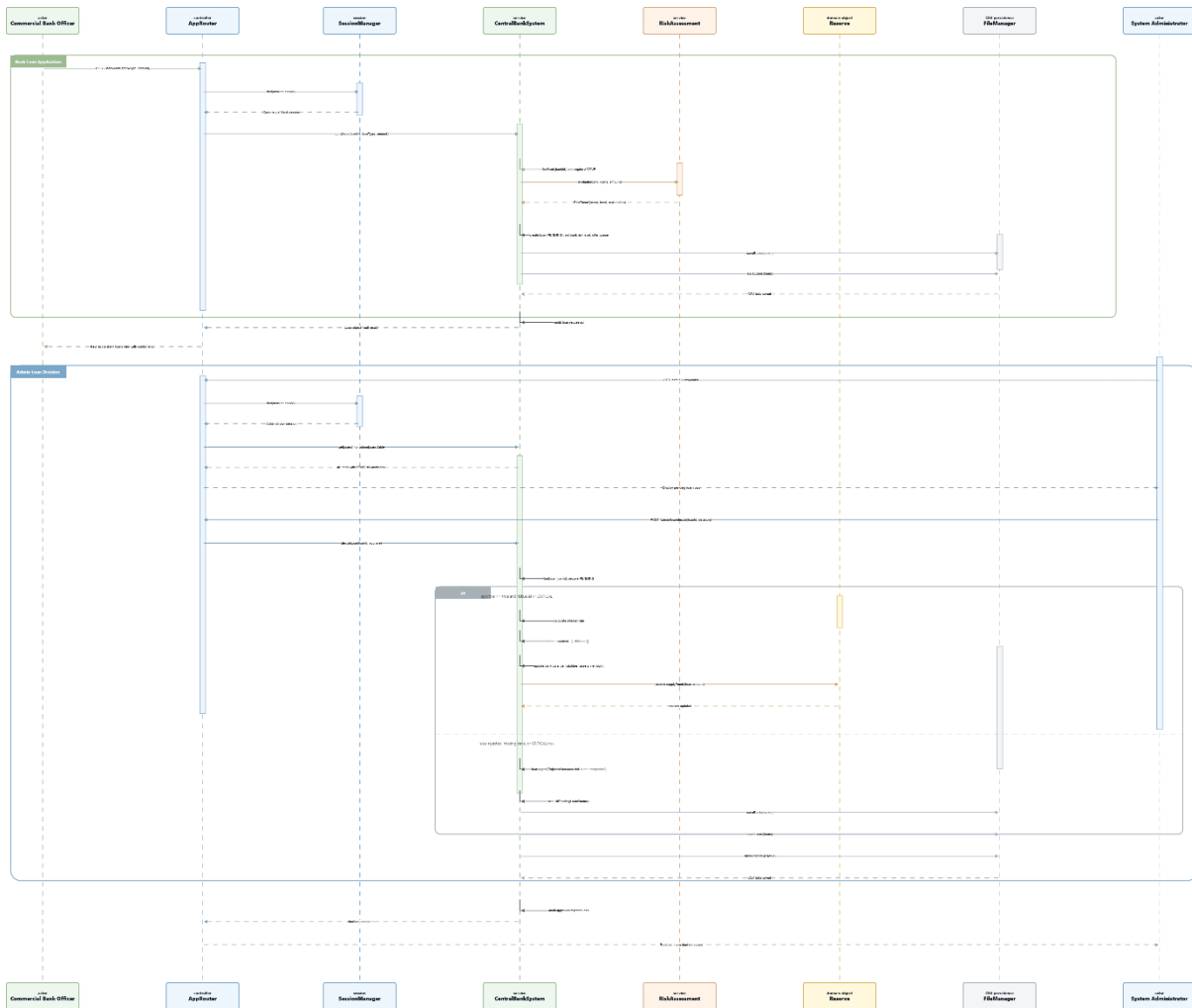


Figure 4: Sequence Diagram of Central Banking System

The System Sequence Diagram shows how an actor interacts with the system during a particular operation. It represents the sequence of requests and responses between the user and the Central Banking System, such as login, processing a loan, making a transaction, requesting approval, or generating a report.

Development Methodology:

For developing the **Central Banking System**, we followed an **Iterative Development Approach**. We selected this approach because the system contains several interconnected modules and each module depends on some common components such as authentication, user roles, data handling, routing, and session management. Instead of developing the complete system at once, we divided the project into smaller and manageable stages. Each module was developed, checked, tested, and improved before moving to the next stage.

At the beginning of the project, we studied the requirements of a central banking management system and identified the major users, operations, data and services that needed to be included. Based on the requirements, we identified two main user categories: **Administrator** and **Commercial Bank**. **Administrator** is the controller who admit and gives permission to **Commercial Banks**. Later, by teacher's suggestion we added **Governor** who controls **Administrator** and stops **Administrator** if **Administrator** is become corrupt.

After identifying the requirements, we designed the overall object-oriented structure of the system. Major Java classes were created to represent the important entities and operations of the banking system. These include **CentralBank**, **CommercialBank**, **FinancialInstitution**, **Governor**, **Administrator**, **Customer**, **Account**, **Loan**, **Transaction**, **Reserve**, **MonetaryPolicy**, **LiquidityOperation**, **ApprovalRequest**, **Notification**, **ExchangeRate** and several supporting classes.

The current version of the project we did not use a traditional relational database. Instead, we used **CSV-based file storage** to maintain system information. Separate CSV files are used for banks, customers, accounts, loans, transactions, administrators, policies, reserves, approval requests, notifications, exchange rates, audit logs and login logs. The **FileManager** and **CsvUtil** classes are responsible for loading and saving this information.

The development was then continued module by module. First Home page which is accessible by anyone. Then login system and for both commercial and admin and later added Governor and whole new separate for login page, also we added live currency exchange rate other little features followed by access control according to their role and the main banking operations. Later, loan management, inter-bank transactions, reserve management, monetary policy, risk assessment, fraud detection, exchange-rate information and reporting features were integrated into the system.

Throughout the development process, we followed an iterative cycle. Whenever a feature was completed, we tested its basic functionality, identified problems and made necessary corrections. This process helped us gradually build a more stable system and reduced the possibility of errors affecting multiple modules at the same time.

The development process consisted of the following stages.

Stage 1: Choosing Project topic and make Project proposal

The first stage of development was to select an appropriate project topic and define the overall idea of the system. We explored several possible project options and evaluated them based on real-world relevance, complexity, and feasibility within the given timeframe. First we wanted to make a bank but because of our course is design project 1 we had to increase difficulty. After careful consideration also with teacher's suggestion, we selected the **Central Banking System** as it represents a complex and practical financial system.

At this stage, we focused on understanding the basic concept of a central banking system and what functionalities such a system should provide. We identified the initial purpose of the project, which includes managing commercial banks, handling financial operations, and maintaining control over the banking environment.

We also prepared the project proposal by outlining the introduction, problem statement, objectives, and expected features of the system. Initial user roles such as **Administrator** and **Commercial Bank** with many others were identified, along with their basic responsibilities. The main system features such as authentication, bank management, transactions, and reporting were also proposed.

The main output of this stage was a clear project idea, defined objectives, initial system idea, and a completed project proposal that provided direction for further development.

Stage 2: Requirement Analysis and Understanding the System

The second stage of development was to understand the problem domain and identify what the proposed Central Banking System needed to accomplish. We studied the basic activities performed by a central banking authority and considered how commercial banks, administrators and the Governor would interact with the system.

At this stage, we identified the major users of the system and defined their responsibilities. The two main user roles were identified as **Administrator and Commercial Bank**. We also identified the main operations that needed to be supported, including commercial bank management, customer and account management, loan processing, inter-bank fund transfer, reserve management, monetary policy management, risk assessment, fraud detection and report generation.

We also considered the information that needed to be stored and managed by the system. This helped us determine the main entities and data structures that would later be represented through Java classes and CSV files.

The main output of this stage was a clear understanding of the system requirements, user roles, major operations and expected system behaviour.

Stage 3: Background Study

The third stage of development focused on conducting a background study of central banking systems and their real-world operations. We researched the roles and responsibilities of central banks, including monetary control, reserve management, financial regulation, and supervision of commercial banks.

At this stage, we analyzed how central banks interact with commercial banks and other financial institutions. We also studied important concepts such as interest rates, inflation control, liquidity management, and financial stability to better understand how these functions could be represented in the system.

We further explored how modern banking systems handle transactions, risk assessment, fraud detection, and reporting. This helped us gain insight into the practical implementation of banking operations and system workflows.

The main output of this stage was a strong understanding of real-world central banking functions, which helped us design a more realistic and effective system model for the project.

Stage 4: UML and System Design

The fourth stage of development focused on designing the structure of the system using UML diagrams. Based on the requirements and background study, we created initial diagrams to represent the system's components and their interactions.

At this stage, we developed key diagrams including the **Use Case Diagram**, **Class Diagram**, **Activity Diagram**, and **Sequence Diagram**. These diagrams helped us visualize user interactions, system workflows, and relationships between different classes and objects.

We also defined the main system entities and their attributes, such as banks, customers, accounts, loans, and transactions. The relationships between these entities were carefully planned to ensure proper data flow and system functionality.

The main output of this stage was a clear visual representation of the system design, which served as a blueprint for the implementation phase.

Stage 5: Learning Web Application Fundamentals

The fifth stage of development focused on understanding how web applications work and how they can be applied to our system. We studied the basic concepts of frontend and backend interaction, including how user requests are processed and how data is displayed through a web interface.

At this stage, we explored important topics such as web page structure, navigation, routing, and basic hosting methods. We also learned how to create and manage simple web pages and how they connect with backend logic.

We applied this knowledge by setting up a basic self-hosted webpage for the project. This helped us understand how the system would be accessed and interacted with by users through a browser.

The main output of this stage was a foundational understanding of web application development and a basic working environment for building the system interface.

Stage 6: Frontend Development and Authentication

The sixth stage of development focused on building the user interface and implementing the authentication system. We started by designing and developing the homepage, which serves as the entry point of the system and is accessible to all users.

At this stage, we created separate login systems for **Administrator** and **Commercial Bank** users. Each login system was designed to validate user credentials and provide access based on their respective roles.

We also worked on structuring the interface to ensure smooth navigation between different pages and features. Basic user interaction elements were added to make the system more usable and organized.

The main output of this stage was a functional frontend with a working login system, allowing different types of users to securely access the system.

Stage 7: Backend Development

The seventh stage of development focused on implementing the backend logic of the system. We developed the core functionalities that handle data processing, system operations, and communication between different modules.

At this stage, we connected the frontend interface with backend functions to ensure that user actions such as login, data entry, and system operations were properly processed. Key features such as account handling, transactions, and basic banking operations were implemented.

We also ensured that the system could store and retrieve data correctly using CSV-based file handling. This helped maintain consistency and accuracy of information across different modules.

The main output of this stage was a functional backend system with connected features, enabling the system to perform core banking operations effectively.

Stage 8: Exchange Rate Integration

The eighth stage of development focused on adding the live exchange rate feature to the system because of Teacher's suggestion. This feature allows users to view up-to-date currency values by using API, which is important for a realistic banking environment.

At this stage, we integrated an external API to fetch real-time exchange rate data. We developed functions to request, receive, and process this data so that it could be displayed within the system.

We also ensured that the exchange rate information was properly updated and stored when necessary, allowing the system to use this data in relevant financial operations.

The main output of this stage was a working exchange rate module that provides live currency data and enhances the overall functionality of the system.

Stage 9: Governor Role Implementation

The ninth stage of development focused on introducing the **Governor** role into the system because of Instructor's reminder we knew that **Administrator** can become corrupt. So, we added this role was added to provide a higher level of authority and control over administrators and overall system operations.

At this stage, we designed and implemented the Governor's functionalities, including monitoring administrators, approving or rejecting important actions, and controlling administrator access when necessary.

We also updated the system structure to support role hierarchy, ensuring that the Governor has higher privileges than the Administrator, while maintaining proper access control across all user roles.

The main output of this stage was the successful integration of the Governor role, adding an additional layer of security, supervision, and control to the system.

Stage 10: System Integration and Debugging

The tenth stage of development focused on integrating all system modules and ensuring they work together correctly. We combined the previously developed features, including authentication, banking operations, exchange rates, and the Governor role, into a single system.

At this stage, we tested the interaction between different modules to ensure smooth data flow and proper functionality. We identified and fixed errors, improved system performance, and ensured that role-based access control was working correctly across all users.

We also refined existing features and made necessary adjustments to improve system stability and usability. Special attention was given to handling errors and ensuring consistent data management.

The main output of this stage was a stable and fully integrated system with minimized errors and improved overall performance.

Stage 11: Final UML Updates

The eleventh stage of development focused on updating and refining all UML diagrams to match the final implemented system. As several features and roles were added during development, the initial diagrams required modification.

At this stage, we revised the **Use Case Diagram**, **Class Diagram**, **Activity Diagram**, and **Sequence Diagram** to accurately represent the system's current structure and behavior. These updates ensured that all user roles, system processes, and interactions were clearly reflected.

We also verified that the relationships between classes, workflows, and operations in the diagrams were consistent with the actual implementation of the system.

The main output of this stage was a complete and accurate set of UML diagrams that fully represent the final version of the Central Banking System.

Stage 12: Final Report and Presentation

The final stage of development focused on preparing the complete project report and presentation. We organized all the work done throughout the development process into a structured document.

At this stage, we compiled all sections of the report, including introduction, objectives, methodology, system design, implementation details, and diagrams. We ensured that all UML diagrams and system explanations accurately reflected the final version of the project.

We also prepared the project presentation, highlighting the key features, system workflow, and important functionalities of the Central Banking System. Efforts were made to clearly demonstrate how the system works and how different user roles interact with it.

The main output of this stage was a complete final report and a well-prepared presentation ready for submission and evaluation.

Implementation:

These are pictures of our completed project first you can see home page, About Us and Login Page which is viewed by everyone. Then in Login Page if you press Administrator it takes you to Administrator login page after you fill up the info it takes it to Administrator Dashboard. Same for Commercial Bank option in login. Then there is Governor Login Page which is completely independent private link. And this page is not linked to anything with home page and after fill up the data's it takes you to Governor Dashboard. Below are the figure of the projects and last is the link to more figures.

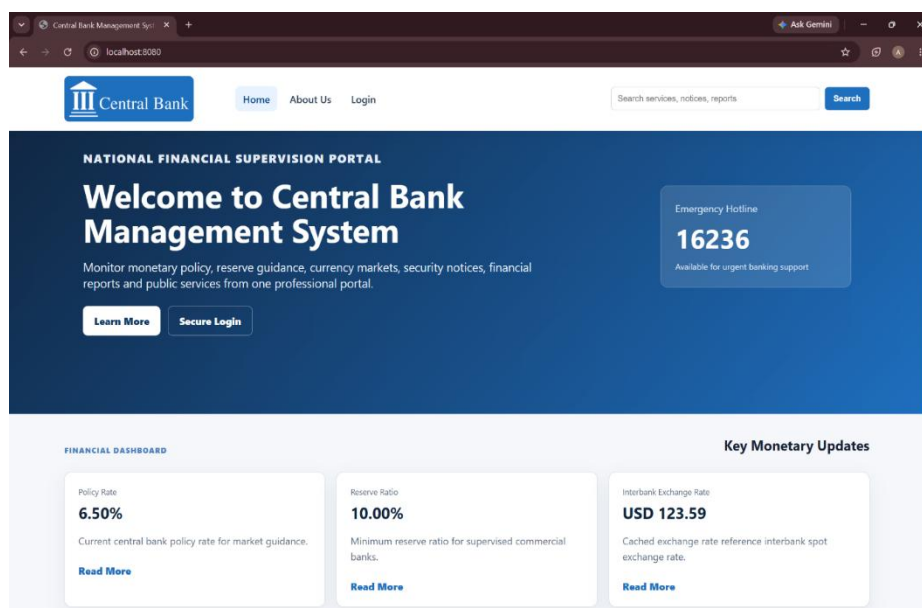


Figure 5: Home Page

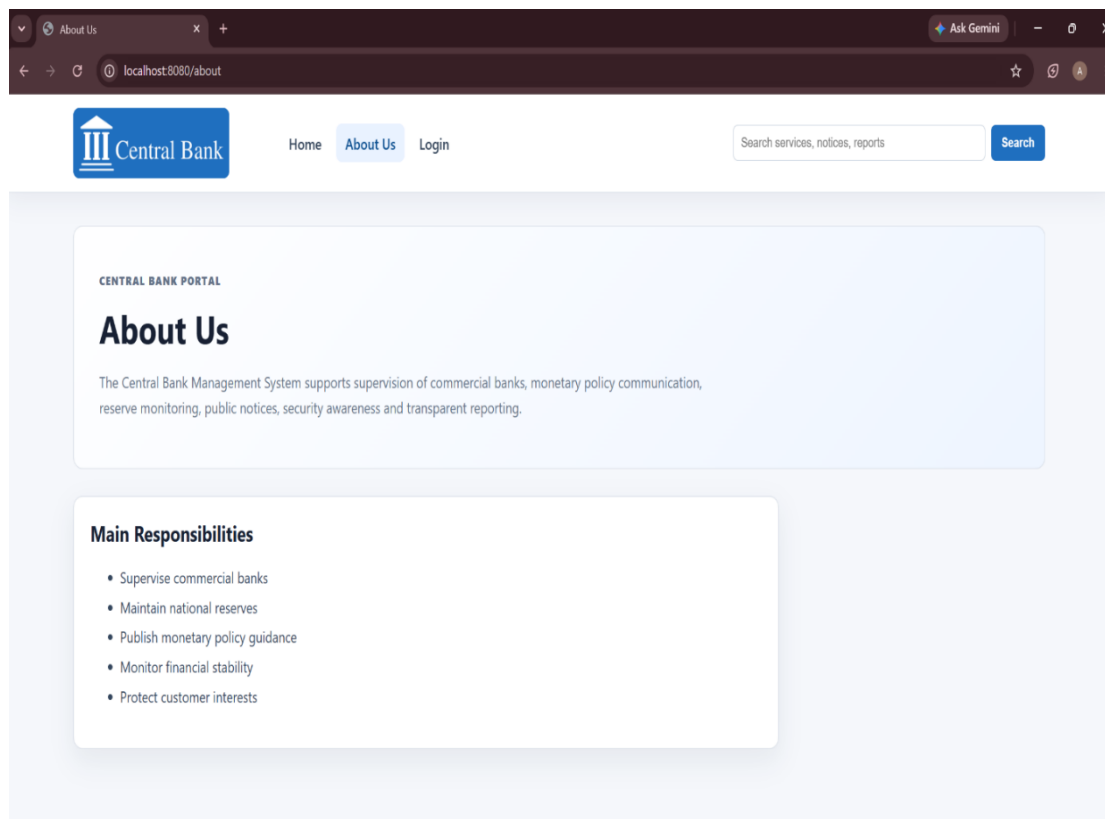


Figure 6: About Us

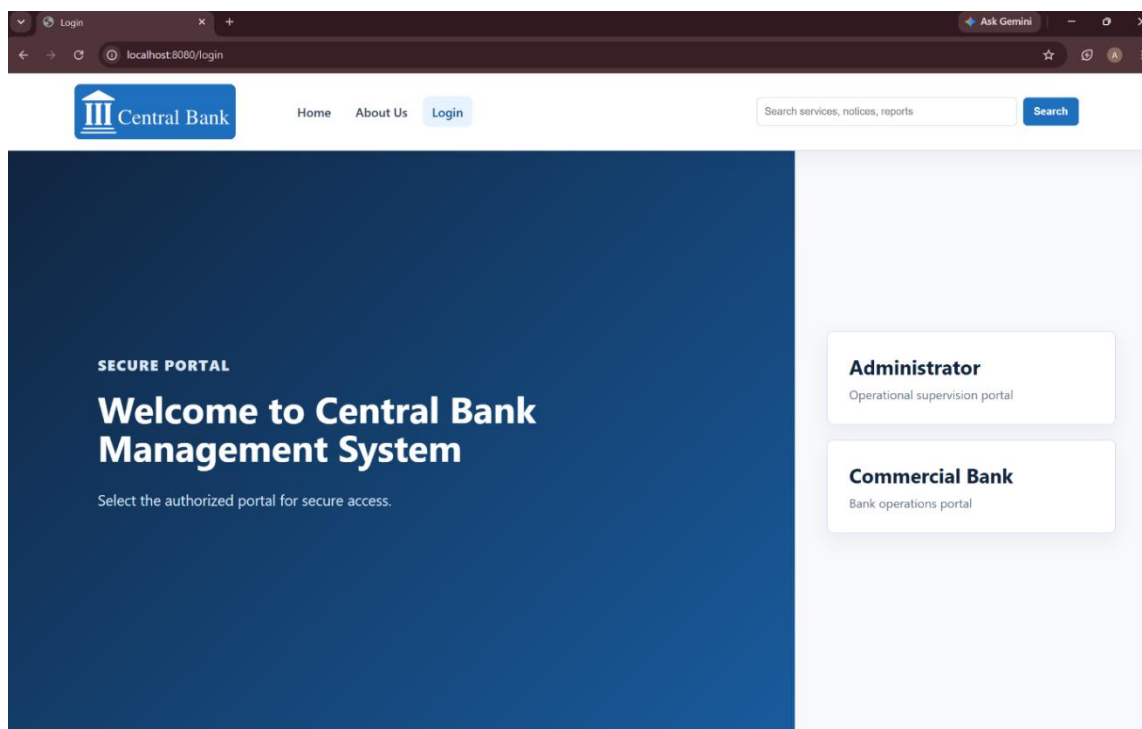


Figure 7: Login Page

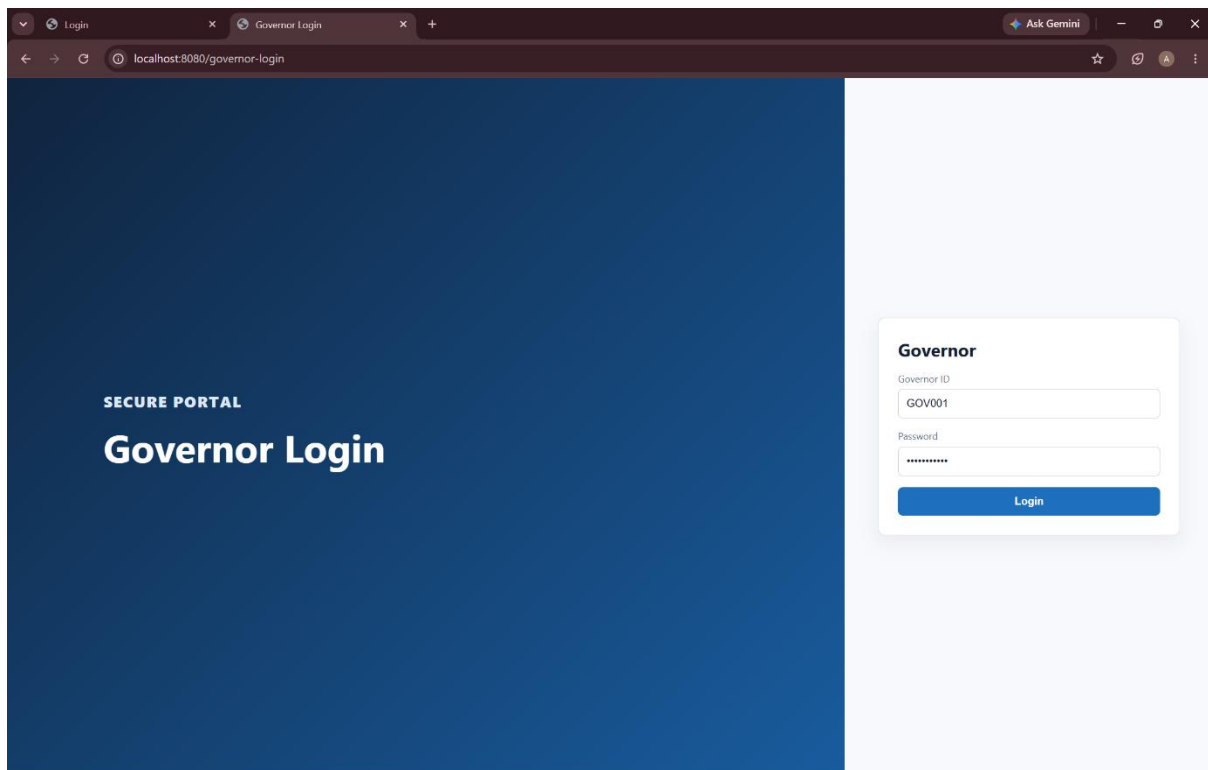


Figure 8: Governor Login

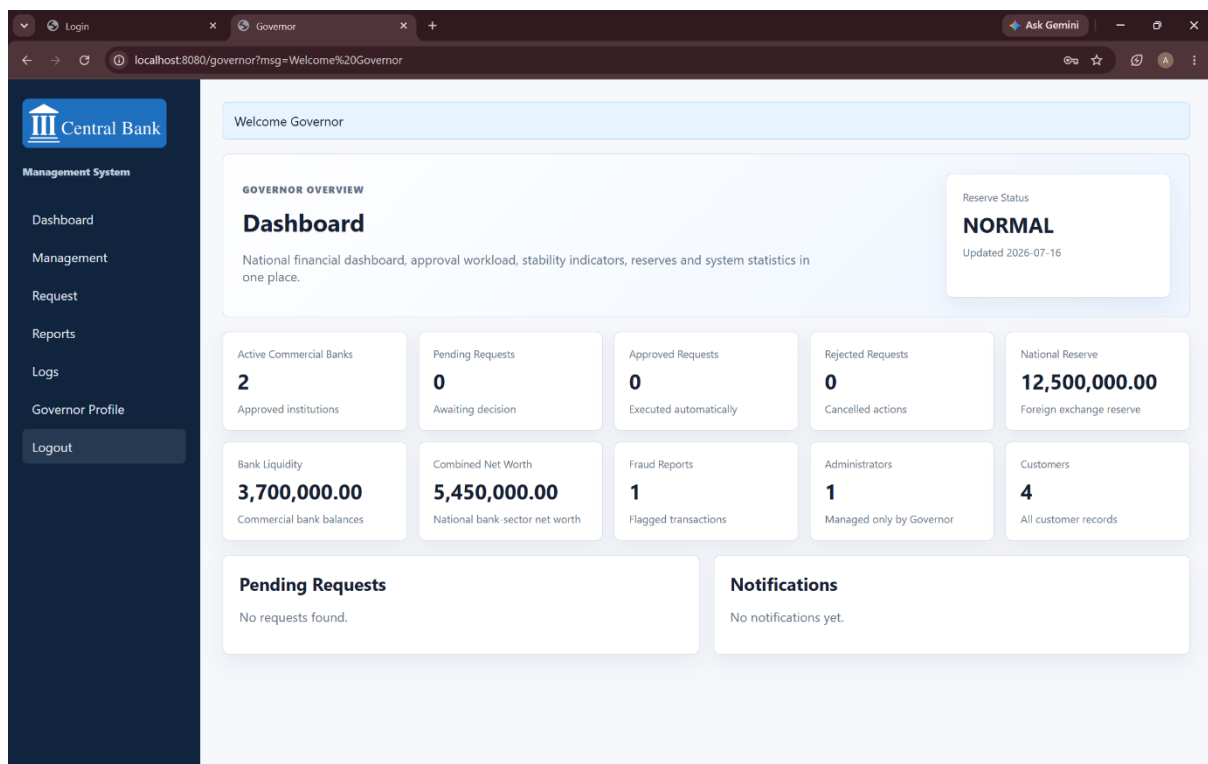


Figure 9: Governor Dashboard

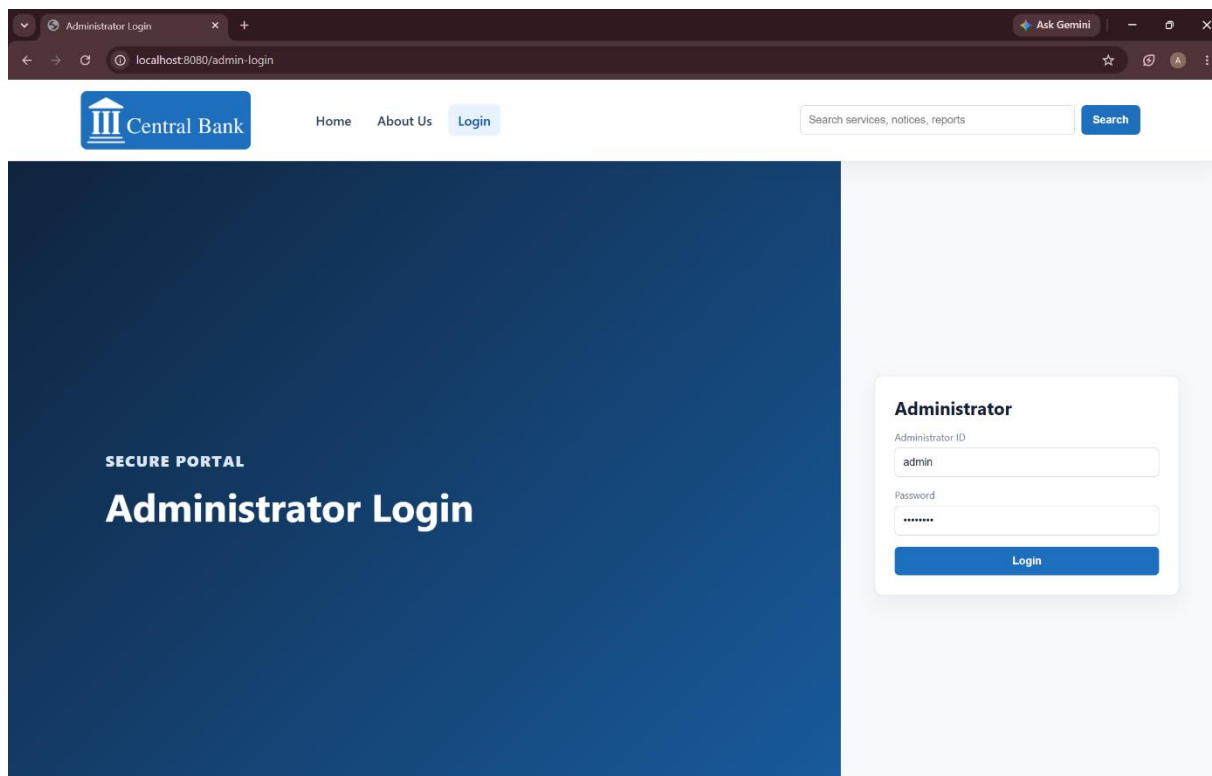


Figure 10: Administrator Login

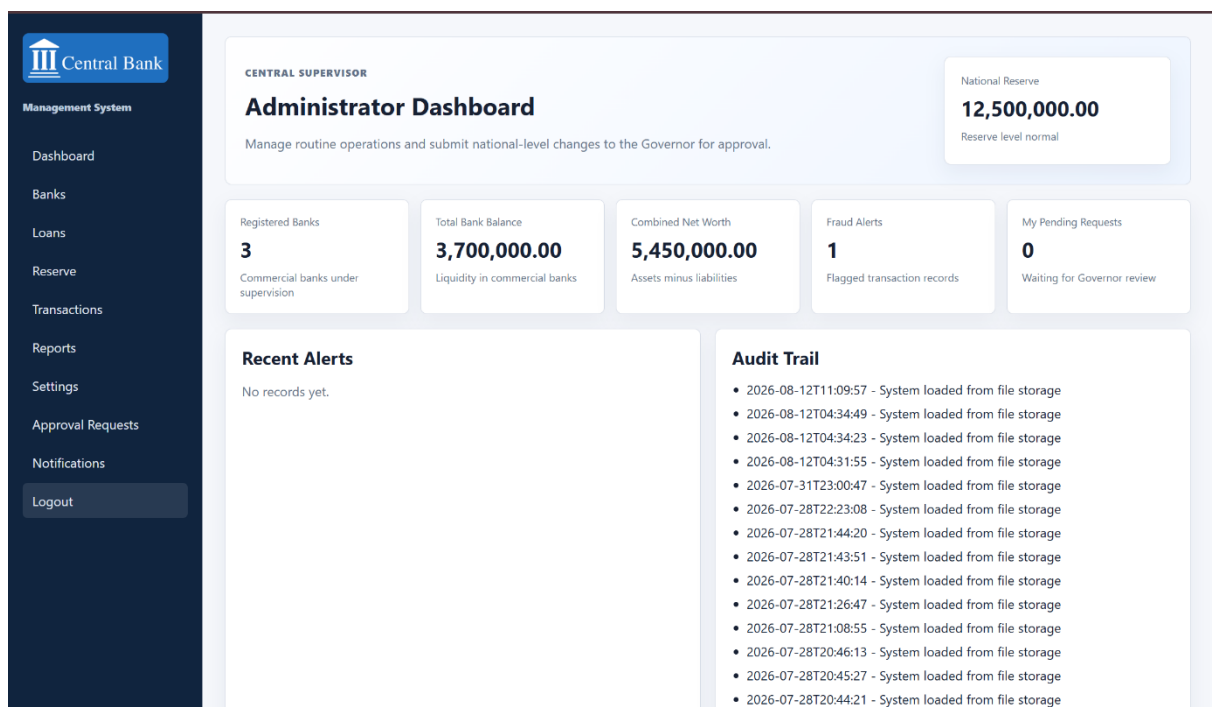


Figure 11: Administrator Dashboard

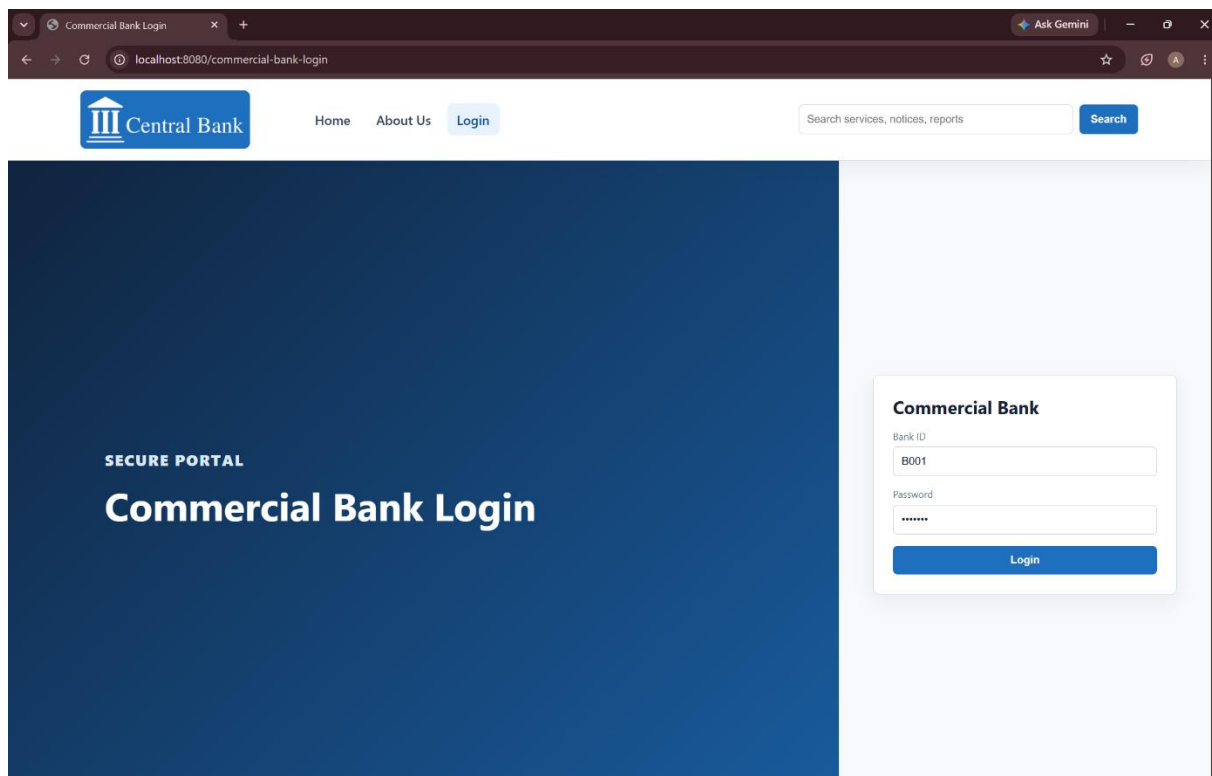


Figure 12: Commercial Bank Login

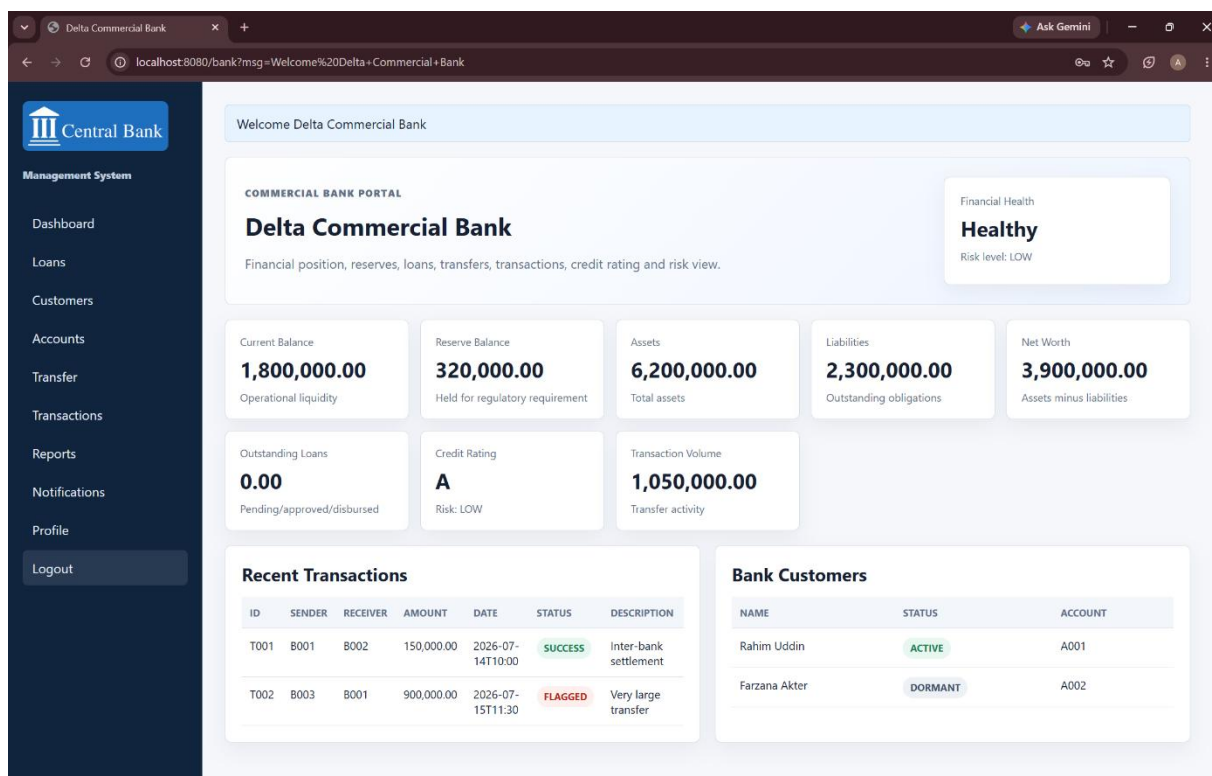


Figure 13: Commercial Bank Dashboard

Below is the Screenshot of project a in google drive link below:

[Google Drive Link of Central Banking System](#)

Conclusion:

The Central Banking System Simulator successfully brings together the major banking management and monitoring functions which defined the project. The implemented system supports commercial bank registration and management, customer and account management, loan processing, inter-bank fund transfers, deposits and withdrawals, reserve management, monetary policy updates, money-supply and liquidity operations, approval workflows, notifications, exchange-rate information, and report generation.

The implemented CentralBankSystem simulator acts as the main controller of these operations which separate classes such as CentralBank, CommercialBank, Customer, Account, Loan, Transaction, and Reserve represent the major banking entities. This project also includes risk assessment, fraud detection, reporting, session management, login management, file management, and a web interface. This demonstrates the application of object-oriented programming and system-design concepts which we described in the proposal.

The system provides a controlled environment for demonstrating central banking operations without directly affecting a real financial system and also creates a simulator that can be used to understand, test, and analyse banking operations.

Overall, the project provides a functional foundation of the Central Banking System Simulator and demonstrates how software can be used to organize banking operations, monitor financial activities and manage approvals.

Future Work & Limitations

Future Work

In future to improve Central Banking System Simulator in following areas:

1. Database integration

Replace the current file-based storage with a centralized relational database to improve scalability, consistency and backup of data.

2. Advanced fraud detection

Improve the existing fraud-detection by using machine-learning which can identify suspicious transactions.

3. Advanced risk analysis

Improve the current risk-assessment mechanism by adding additional financial indicators, bank performance, and market conditions.

4. Security & System Alerts

Introduce multi-factor authentication, encryption, security auditing, and improve protection against unauthorized access on this system. Add notifications for approval requests, transactions, alerts, and other important banking activities.

5. Advanced dashboards and visualization

Add interactive charts and analytical dashboards of the bank performance, transaction volume, loan status, reserves, fraud alerts, and financial stability so that customers can detect it easily.

6. Scalability and performance testing

Test the system with larger datasets and higher transaction volumes which identify the performance limitations and handle larger banking environments.

These improvements will take the central banking simulator from a more advanced platform.

Limitations

This system implements a wide range of banking functions which remains a simulation system rather than a real central banking platform. The financial operations, risk calculations and fraud detection are simplified.

The system also depends on locally stored data and file-based information. The project contains CSV data files for banks, customers, accounts, transactions, loans, reserves, operations, policies, approval requests, notifications, administrators, and other features. Therefore, In the future it could be used as a more robust database architecture for larger-scale data management systems.

Another limitation is the complexity of fraud and risk analysis systems. The project includes FraudDetector and RiskAssessment but real financial institutions require larger datasets, more sophisticated analytical models, and continuous monitoring to identify all the complex fraudulent activities which will monitor the whole system. This would be particularly relevant because transaction monitoring and fraud detection were identified as important challenges in the proposal.

Security is another limitation of this project. The current system includes login, sessions, user roles, administrator management, and audit/login/activity logs, but a real central banking platform would require much protected security controls, encryption, and regulatory compliance mechanisms.

Lastly the system is not designed to handle a large number of transactions and maintain high availability and reliability as like the Central bank.

Reference:

- Liang, Y. Daniel. Introduction to Java Programming and Comprehensive Version 10th edition. Pearson.
- Schildt, Herbert. Java: The Complete Reference, 11th Edition. McGrawHill Education.
- Course Notes and Presentations of Object-Oriented Programming, Structured Programming, Introduction to Programming
- Online tutorials for Java Swing and file handling
- YouTube Video URL: <https://www.youtube.com/watch?v=dwdFRP1ipY>, <https://www.youtube.com/watch?v=Y4QbLV7M8gw>
- Bangladesh Bank official website and related resources
- Central bank. (2026). In *Wikipedia*. Retrieved June 21, 2026, from https://en.wikipedia.org/wiki/Central_bank^[1]